Lab No: 02

Student Name: Hania Nawab

Roll No: 17

Task 1:	

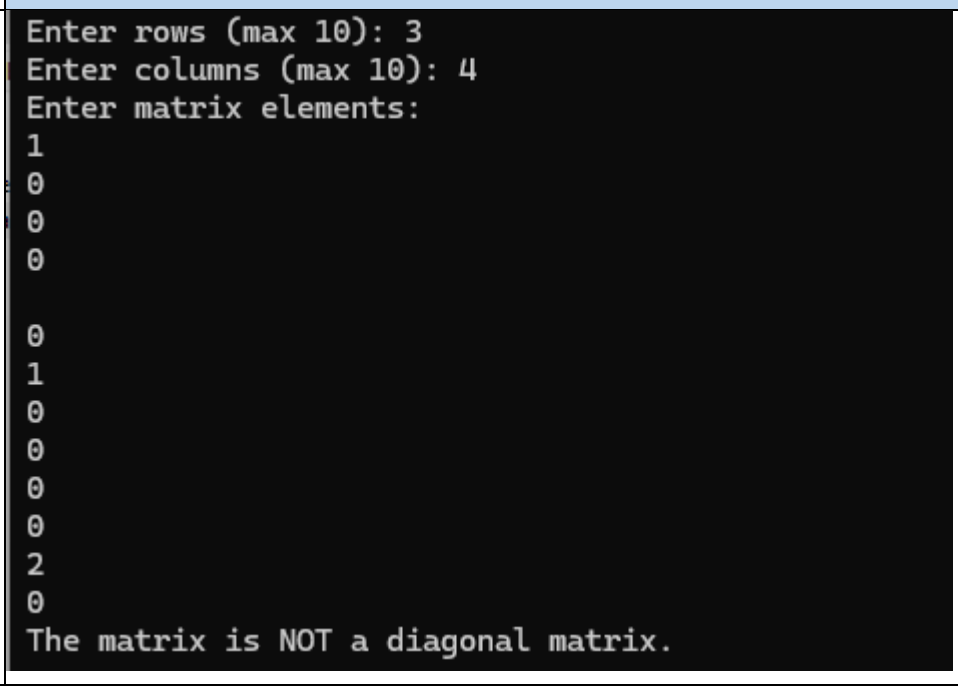
Code:	#include <iostream>
	using namespace std;
	
	int main() {
	  int m[10][10];
	  int rows, cols;
	
	  cout << "Enter rows (max 10): ";
	  cin >> rows;
	  cout << "Enter columns (max 10): ";
	  cin >> cols;
	
	  cout << "Enter matrix elements:" << endl;
	  for (int i = 0; i < rows; i++) {
	  for (int j = 0; j < cols; j++) {
	  cin >> m[i][j];
	  }
	  }
	
	  bool diagonal = true;
	
	  if (rows != cols) {
	  diagonal = false; // must be a square matrix
	  } else {
	  for (int i = 0; i < rows; i++) {
	  for (int j = 0; j < cols; j++) {
	  if (i != j && m[i][j] != 0) {

```
                diagonal = false;
            }
        }
    }
}

if (diagonal)
    cout << "The matrix is a diagonal matrix." << endl;
else
    cout << "The matrix is NOT a diagonal matrix." << endl;

return 0;
}
```

Screenshots:



Task 2:

Code:

```
#include <iostream>
using namespace std;

int* arr = new int[2];
int n = 0;
int cap = 2;

void resizeArray() {
```

```
    cap = cap * 2;
    int* temp = new int[cap];
    for (int i = 0; i < n; i++) {
        temp[i] = arr[i];
    }
    delete[] arr;
    arr = temp;
}

void insertValue(int pos, int val) {
    if (pos < 0 || pos > n) {
        cout << "Boundary Error" << endl;
        return;
    }
    if (n == cap) {
        resizeArray();
    }
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = val;
    n++;
    cout << "Value inserted." << endl;
}

void updateValue(int pos, int val) {
    if (pos < 0 || pos >= n) {
        cout << "Boundary Error" << endl;
        return;
    }
    arr[pos] = val;
    cout << "Value updated." << endl;
}

void deleteValue(int pos) {
    if (pos < 0 || pos >= n) {
        cout << "Boundary Error" << endl;
        return;
    }
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;
    cout << "Value deleted." << endl;
}
```

```
void searchValue(int val) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == val) {
            cout << "Found at index " << i << endl;
            return;
        }
    }
    cout << "Value not found." << endl;
}

void displayArray() {
    if (n == 0) {
        cout << "Array is empty." << endl;
        return;
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int choice, pos, val;

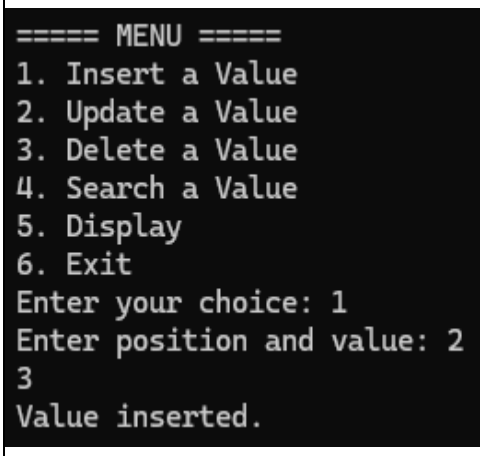
    do {
        cout << "\n===== MENU =====" << endl;
        cout << "1. Insert a Value " << endl;
        cout << "2. Update a Value" << endl;
        cout << "3. Delete a Value" << endl;
        cout << "4. Search a Value" << endl;
        cout << "5. Display" << endl;
        cout << "6. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

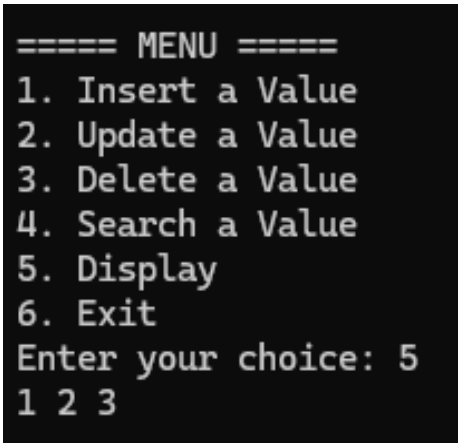
        if (choice == 1) {
            cout << "Enter position and value: ";
            cin >> pos >> val;
            insertValue(pos, val);
        } else
        if (choice == 2) {
            cout << "Enter position and new value: ";
            cin >> pos >> val;
            updateValue(pos, val);
        } else
        if (choice == 3) {
```

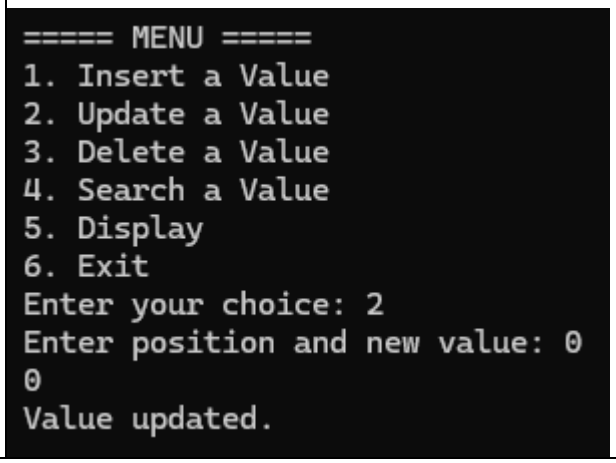
```
            cout << "Enter position to delete: ";
            cin >> pos;
            deleteValue(pos);
        } else
        if (choice == 4) {
            cout << "Enter value to search: ";
            cin >> val;
            searchValue(val);
        } else
        if (choice == 5) {
            displayArray();
        } else
        if (choice != 6) {
            cout << "Invalid choice." << endl;
        }
    } while (choice != 6);

    delete[] arr;
    return 0;
}
```

Screenshot:







```
===== MENU =====
1. Insert a Value
2. Update a Value
3. Delete a Value
4. Search a Value
5. Display
6. Exit
Enter your choice: 5
0 2 3

===== MENU =====
1. Insert a Value
2. Update a Value
3. Delete a Value
4. Search a Value
5. Display
6. Exit
Enter your choice: 4
Enter value to search: 1
Value not found.
```

Task 3:	

Code:	#include <iostream>
	using namespace std;
	
	bool friends[5][5] = {
	{false, true, false, true, true }, // 0
	{true, false, true, false, true }, // 1
	{false, true, false, false, false}, // 2
	{true, false, false, false, true }, // 3
	{true, true, false, true, false} // 4
	};
	
	bool commonFriend(int a, int b) {
	for (int k = 0; k < 5; k++) {
	if (friends[a][k] && friends[b][k]) {
	return true;

```
        }
    }
    return false;
}

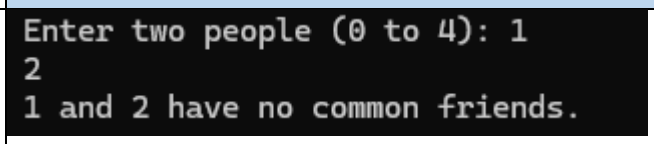
int main() {
    int a, b;
    cout << "Enter two people (0 to 4): ";
    cin >> a >> b;

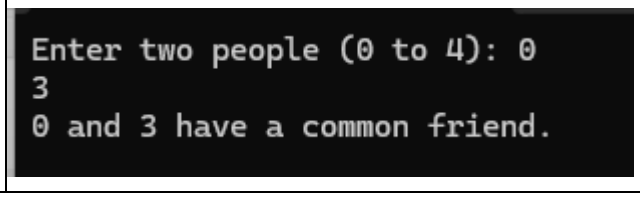
    if (a < 0 || a > 4 || b < 0 || b > 4) {
        cout << "Boundary Error" << endl;
        return 0;
    }

    if (commonFriend(a, b))
        cout << a << " and " << b << " have a common friend." << endl;
    else
        cout << a << " and " << b << " have no common friends." << endl;

    return 0;
}
```

Screenshots:





Task 4:

Code:

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    string names[5] = {"Ali", "Hiba", "Asma", "Zain", "Faisal"};
    int subjects[5] = {5, 4, 3, 3, 5};
    int creditHours = 3;

    double ali[5]    = {3.66, 3.33, 4.0, 3.0, 2.66};
    double hiba[4]   = {3.33, 3.0, 3.66, 3.0};
    double asma[3]   = {4.0, 3.66, 2.66};
    double zain[3]   = {2.66, 2.33, 4.0};
    double faisal[5] = {3.33, 3.66, 4.0, 3.0, 3.33};
    double* source[5] = {ali, hiba, asma, zain, faisal};

    // jagged array: each row has its own size
    double** grades = new double*[5];
    for (int i = 0; i < 5; i++) {
        grades[i] = new double[subjects[i]];
        for (int j = 0; j < subjects[i]; j++) {
            grades[i][j] = source[i][j];
        }
    }

    for (int i = 0; i < 5; i++) {
        double totalPoints = 0;
        int totalCredits = 0;
        for (int j = 0; j < subjects[i]; j++) {
            totalPoints = totalPoints + grades[i][j] * creditHours;
            totalCredits = totalCredits + creditHours;
        }
        double gpa = totalPoints / totalCredits;
        cout << names[i] << " GPA: " << gpa << endl;
    }

    for (int i = 0; i < 5; i++) {
```

```
        delete[] grades[i];
    }
    delete[] grades;

    return 0;
}
```

Screenshots:

